Achados até a apresentação preliminar (02/11/2026)

Coisas que **não** esperávamos e que só apareceram porque coletamos evidência e comparamos com a previsão. Cada achado separa **fato** (o que o log/métrica mostra) de **inferência** (nossa explicação) e diz o que foi feito.

#	Achado	Gravidade	Situação
1	Sonda do watchdog levava 2,8–3,6 s apesar de `TIMEOUT=2`	Média	**Corrigido**
2	Alerta `WatchdogFlapping` disparou com só 2 ciclos DOWN→UP	Baixa	**Corrigido**
3	A 3ª falha que tira o nó do pool veio do teste sintético, não do usuário	Informativo	Documentado
4	Watchdog vê o nó UP em ~2–3 s, mas o tráfego leva até ~10 s para voltar	Média	Documentado; experimento na Hora 6
5	`TaxaErro5xxAlta` disparou sem nenhum erro chegar ao usuário	Alta (falso positivo)	**Corrigido** (parcial)
6	Liveness do Nginx dependia dos nós	Média	**Corrigido**
7	Relógio do host (WSL2) salta −2,4 s / +5,1 s	Média	Documentado; mitigado
8	Nó volta a ficar pronto, mas o Nginx ainda o exclui por até 10 s	Média	Documentado (encontrado em 05/11 na revisão das evidências); script corrigido

---

1. Timeout da sonda não cobria o DNS

- **Fato** (`evidencias/preliminar/05b_ANTES_correcao_watchdog_dns.txt`): com o rust- a parado, `erro=URLError:[Errno_-2]_Name_does_not_resolve ... latencia_ms=3636` (e 2847, 2844). `TIMEOUT=2`.
- **Inferência**: o `timeout` do `urllib` vale para conexão e leitura, não para a resolução de nome. Com o contêiner parado o DNS do Docker não tem mais o registro e demora ~3 s para responder "não existe".
- **Por que importa**: a sonda ultrapassava o próprio prazo; com vários alvos lentos o ciclo de 5 s escorregaria e a idade da última sondagem cresceria.
- **Correção** (`watchdog/watchdog.py`, `sondar`): a requisição roda numa thread e a sonda desiste no prazo total. Depois: `erro=prazo_total_2s_excedido(dns/conexao/leitura) latencia_ms=2002` (`05_no_parado_e_recuperacao.txt`).

2. Falso positivo de flapping por extrapolação

- **Fato**: `WatchdogFlapping` (`increase(watchdog_state_transitions_total[10m]) > 4`) ficou `firing` após **2** ciclos DOWN→UP (= 4 transições) (`05b`).
- **Inferência**: `increase()` extrapola o contador até as bordas da janela, então 4 vira > 4.
- **Correção**: `changes(watchdog_target_up{kind="readiness"}[10m]) >= 6` (3 ciclos completos; `changes` conta mudanças reais, sem extrapolar).

3. O teste sintético participa do `max_fails`

- **Fato** (error log do Nginx em `05b`): das 3 falhas que levaram a `upstream server temporarily disabled`, duas tinham `client: 172.20.0.1` (o curl do host) e uma `client: 172.20.0.6` (o watchdog fazendo o teste sintético via Nginx).
- **Inferência**: o tráfego sintético entra no mesmo pool e no mesmo contador. Por isso só **2** requisições do usuário pagaram o timeout de ~1 s, e não 3.
- **Leitura**: é bom (o sintético "absorve" parte do custo), mas o número de requisições lentas que o usuário vê **depende do volume de sintético** — e o sintético também vira carga extra no nó restante. Isso precisa constar na matriz de experimentos.

4. Recuperação: o nó volta antes do tráfego

- **Fato** (`11_tempo_retorno_trafego.txt`): após `docker compose start rust-a`, o watchdog registrou `transicao=UP` em ~2,2 s, mas a primeira requisição de usuário atendida por rust-a veio em **10,5 s** (rodada 1) e **9,4 s** (rodada 2). No intervalo, o error log mostra `rust-a could not be resolved`. Porém, em `05_no_parado_e_recuperacao.txt` o tráfego já tinha voltado **≤ 4,6 s** após o start (start 23:23:53.61; 1ª resposta de rust-a 23:23:58.22). O título daquela seção ("o Nginx ainda NÃO") era a previsão, e foi **refutada**.
- **Inferência**: com `server rust-a:8080 resolve` e `resolver 127.0.0.11 valid=10s`, quando o contêiner para o nome some do DNS e o Nginx tira o servidor do upstream; ele só volta na próxima re-resolução. Como a re-resolução ocorre num ciclo de 10 s que não está sincronizado com o `start`, o atraso fica **entre 0 e ~10 s** conforme a fase do ciclo (3 medições: ≤ 4,6 s, 9,4 s, 10,5 s). O `fail_timeout=10s` também contribui.
- **Por que importa**: watchdog e Nginx **discordam** por até ~8 s. Para o incidente da Hora 7, "o watchdog diz UP" não significa "o nó está recebendo tráfego".
- **Próximo passo** (Hora 6): repetir com `valid=5s` e sem `resolve` e comparar o tempo de retorno (trade-off: mais consultas DNS × retorno mais rápido; sem `resolve`, um IP novo após recriação não seria encontrado).

5. Falso positivo no SLI de erro

- **Fato** (`05c_ANTES_correcao_sli_erro_e_janela_retorno.txt`): `TaxaErro5xxAlta` em `firing`. No mesmo período, o teste com rust-a não pronto mostrou **10/10 respostas 200 ao usuário** (`03_health_checks.txt`).
- **Inferência**: o nó não pronto responde 503 em `GET /`; o Nginx refaz em B; mas o 503 já foi contado em `app_requests_total{status="503"}` no nó. A métrica mede **tentativas no nó**, não **respostas ao usuário**. Com pouco tráfego, 3 recusas bastam para passar de 2%.
- **Correção**: o numerador exclui `status="503"` (recusas por readiness).
- **Limitação que permanece**: a forma correta é medir na borda (Nginx). Sem exporter do Nginx, um 502 gerado pelo próprio Nginx não entra no SLI. Melhoria priorizada para 12/11.

6. Healthcheck do Nginx dependia dos backends

- **Fato** (error log em `11_tempo_retorno_trafego.txt`, rodada 1): `request: "GET /health/live" ... upstream: "http://172.20.0.2:8080/health/live", client: 127.0.0.1`.

- **Inferência**: o healthcheck do Docker no Nginx chamava `/health/live`, que o Nginx repassava aos nós. Se A e B caíssem, o Nginx seria marcado `unhealthy` sem ter problema — o mesmo erro do exemplo do enunciado ("se /health/live dependesse do banco...").
- **Correção**: `location = /nginx/live { return 200; }` respondido pelo próprio Nginx.

7. Relógio do host dá saltos

- **Fato** (`10_relogio_wsl.txt`): `time.time()` comparado ao relógio monotônico mostra saltos periódicos de **−2,4 s** e **+5,1 s** (~a cada 30 s), no host e dentro do contêiner. Visível também em `03_health_checks.txt` da 1ª rodada (seção com horário anterior à seção que a precede).
- **Inferência**: correção de horário do WSL2; contêineres compartilham o kernel, então o salto atinge todos os serviços ao mesmo tempo.
- **Consequências**: ordenar logs de serviços diferentes só por horário pode inverter eventos; `time() - última_sondagem` pode errar em até ~5 s.
- **Mitigação**: correlacionar por `request_id`; latências medidas com relógio monotônico (watchdog e Rust já fazem); limiar de `MonitorParado` (30 s) com folga sobre 5 s.

8. O nó volta a ficar pronto, mas o Nginx continua a excluí-lo

Encontrado em 05/11, ao revisar as evidências para a apresentação.

- **Fato** (`04_balanceamento_20req.txt`): o título diz "os dois nós de pé", mas as 20 requisições foram **todas para rust-b** (23:23:20.8–21.1). Logo depois, em `05`, as 20 requisições com rust-a parado não tiveram **nenhuma** requisição lenta (~1 ms cada), ao contrário de `05b`/`05c` (2–3 a ~1,09 s).
- **Fato** (error log em `05`): `23:23:13 … upstream server temporarily disabled … client: 172.20.0.6` (o watchdog, teste sintético), com `upstream_status=503, 200`. A prontidão de rust-a foi restaurada (`ready=1`) por volta de 23:23:13.6 (`03`), praticamente no mesmo instante.
- **Inferência**: durante o teste de readiness (`03`), a sonda sintética passa pelo Nginx a cada 5 s e recebe 503 de rust-a, o que **renova** a exclusão do `fail_timeout=10s`. A última falha (23:23:13.67) manteve rust-a fora do pool até ~23:23:23.7. As evidências `04` (23:23:20) e o início de `05` (23:23:22) caíram dentro dessa janela.
- **Por que importa**: "o nó voltou a ficar pronto" não significa "o nó voltou ao balanceamento". É a mesma discordância monitor × balanceador do achado 4, por outro mecanismo (verificação passiva do Nginx).
- **Correção**: o 10/10 citado na apresentação vem de `05` (bloco final) e `05b`, não de `04`. O script `scripts/evidencias_preliminar.sh` agora espera 12 s após restaurar a prontidão antes de medir o balanceamento. Os arquivos de 02/11 foram mantidos como estão, porque são evidência do que de fato aconteceu.